

STRUKDAT BAB 9

Graf (Graph)

1. Pengertian

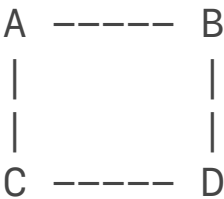
Graf adalah struktur data non-linear yang terdiri dari:

Vertex (Node)

Edge (Sisi)

yang digunakan untuk merepresentasikan hubungan antar objek.

Contoh:



Pada graf:

A, B, C, D

adalah:

Vertex

Sedangkan:

AB

AC

BD

CD
adalah:
Edge

2. Komponen Graf

Vertex

Simpul pada graf.

Contoh:

A B C D E

Edge

Hubungan antar vertex.

Contoh:

A–B

B–C

C–D

3. Graf Berarah (Directed Graph)

Edge memiliki arah.

Contoh:

A → B

B → C

C → D

Representasi:

A → B → C → D

4. Graf Tidak Berarah (Undirected Graph)

Tidak memiliki arah.

Contoh:

A ----- B

B ----- C

C ----- D

5. Graf Berbobot (Weighted Graph)

Setiap edge memiliki bobot.

Contoh:

A --5-- B

A --8-- C

B --2-- C

Bobot dapat berarti:

Jarak

Biaya

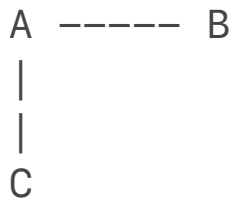
Waktu

Kapasitas

6. Degree

Degree adalah jumlah edge yang terhubung ke vertex.

Contoh:



Degree:

$$A = 2$$

$$B = 1$$

$$C = 1$$

7. In Degree dan Out Degree

Pada graf berarah.

Contoh:



Vertex:

B

memiliki:

$$\text{In Degree} = 1$$

$$\text{Out Degree} = 1$$

8. Representasi Adjacency Matrix

Misal:

A --- B
|
|
C

Matriks:

	A	B	C
A	0	1	1
B	1	0	0
C	1	0	0

Implementasi:

```
int graph[100][100];
```

9. Representasi Adjacency List

Contoh:

A → B → C

B → A

$C \rightarrow A$

Representasi:

A : B C

B : A

C : A

Lebih hemat memori.

10. Traversal DFS

Depth First Search.

Menjelajah sedalam mungkin terlebih dahulu.

Contoh:

A
|
B
|
C
|
D

Hasil:

A B C D

Implementasi:

```
void dfs(int u){

    visited[u] = 1;

    printf("%d ",u);

}
```

11. DFS Rekursif

```
void dfs(int u){

    visited[u] = 1;

    printf("%d ",u);

    for(int v=0;
        v<n;
        v++){

        if(graph[u][v]
            &&
            !visited[v]){
```

```
dfs(v);
```

```
}
```

```
}
```

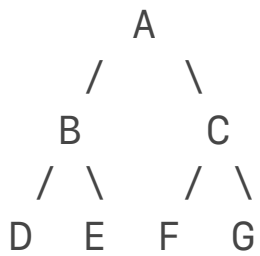
```
}
```

12. Traversal BFS

Breadth First Search.

Per level.

Contoh:



Hasil:

A B C D E F G

Menggunakan Queue.

13. BFS Implementasi

```
void bfs(int start){  
  
    queue[rear++] = start;  
  
    visited[start] = 1;  
  
}
```

Proses:

Ambil Front

Masukkan Tetangga

Ulangi

14. Path

Path adalah jalur antar vertex.

Contoh:

A → B → D

Panjang path:

2 edge

15. Connected Graph

Graf terhubung.

Semua vertex dapat dicapai.

Contoh:

A --- B --- C

16. Disconnected Graph

Graf tidak terhubung.

Contoh:

A --- B

C --- D

Terdapat:

2 komponen

17. Cycle

Siklus.

Contoh:

A → B → C → A

Terdapat jalur kembali ke awal.

18. Kompleksitas

Adjacency Matrix:

Operasi	Kompleksitas
Cek Edge	$O(1)$
DFS	$O(V^2)$
BFS	$O(V^2)$

Adjacency List:

Operasi	Kompleksitas
DFS	$O(V+E)$
BFS	$O(V+E)$

19. Kesalahan Umum

Tidak Menggunakan Visited

Salah:

```
dfs(v);
```

tanpa:

```
visited[v]
```

Akibat:

Infinite Loop

BFS Tanpa Queue

Traversal menjadi salah.

Salah Membuat Edge Dua Arah

Graf:

A --- B

harus:

```
graph[a][b] = 1;
```

```
graph[b][a] = 1;
```

Vertex Di Luar Batas

Salah:

```
graph[1000][1000]
```

padahal:

```
MAX = 100
```

20. Implementasi Lengkap

```
#include <stdio.h>
```

```
int graph[100][100];
```

```
int visited[100];

int n;

void dfs(int u){

    visited[u] = 1;

    printf("%d ",u);

    for(int v=0;
        v<n;
        v++){

        if(graph[u][v]
            &&
            !visited[v]){

            dfs(v);

        }

    }

}

int main(){

    n = 4;

    graph[0][1] = 1;
    graph[1][0] = 1;

    graph[1][2] = 1;
    graph[2][1] = 1;

    graph[2][3] = 1;
```

```
graph[3][2] = 1;  
  
dfs(0);  
  
return 0;  
  
}
```
